Machine Learning Applied to a DC Motor

Dave Figueroa

June 2025

1 Introduction

The precise control of DC motors is critical in numerous applications ranging from robotics and automation to electric vehicles and industrial machinery [1, 2]. Traditional control methods, such as proportional-integral-derivative (PID) controllers, are widely used but often face limitations in handling complex, nonlinear dynamics and varying operational conditions. These limitations motivate the exploration of advanced control strategies, such as reinforcement learning (RL), capable of adapting autonomously to changing conditions and uncertainties [3].

In this context, this project investigates the application of deep deterministic policy gradient (DDPG), a state-of-the-art RL technique. DDPG is particularly well-suited for continuous action spaces, making it an ideal candidate for controlling DC motors, where the control input is a continuous variable [4]. The goal of this project is to develop a RL agent that learns to control the armature voltage applied to a DC motor in order to track a dynamically varying reference velocity. The agent will be trained using simulated data generated from a custom MATLAB SIMULINK model of the DC motor, which captures the essential dynamics and characteristics of the system.

The subsequent sections detail the experimental approach, implementation specifics, and the comprehensive evaluation of the trained agent. The results highlight the advantages of RL in addressing practical challenges inherent in DC motor control, providing both theoretical insights and practical validation of the DDPG framework.

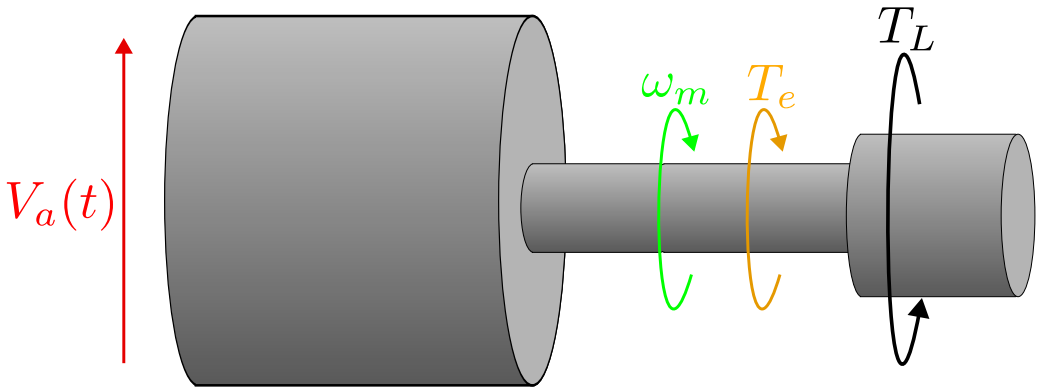


Fig. 1: DC Motor and its load

2 DC Motor Model

2.1 Mathematical Model

A DC motor can be modeled in terms of its electrical and mechanical components [2]. The electrical part consists of the armature circuit, which includes the armature inductance, resistance, and back electromotive force (EMF). The equations governing the electrical dynamics of the motor are given by:

$$v_a(t) = L_A \frac{di_a(t)}{dt} + R_A i_a(t) + v_{\text{rot}}(t) \tag{1}$$

where L_A is the armature inductance, R_A is the armature resistance, $V_a(t)$ is the armature voltage of the motor and $i_a(t)$ is the armature current. The back EMF, which is proportional to the angular velocity of the motor, can be expressed as:

$$v_{\text{rot}}(t) = G_{fq} i_f(t) \omega_m(t) \tag{2}$$

In this equation, G_{fq} is the field-to-rotor flux linkage constant, $i_f(t)$ is the field current, which will be considered constant ($i_f(t) = I_F$), and $\omega_m(t)$ is the angular velocity of the motor.

The mechanical part of the motor is described by the torque equation:

$$T(t) = T_e(t) - T_L(t) \tag{3}$$

where $T_e(t)$ is the electrical torque and $T_L(t)$ is the load torque. The electrical torque can be expressed as

$$T_e(t) = G_{fq} I_F i_a(t) \tag{4}$$

and the load torque is modeled as a function of the motor speed $\omega_m(t)$ and a constant B representing the load characteristics. Thus, we have

$$T_L(t) = B\omega_m(t) \tag{5}$$

The relationship between the torque and the angular velocity is given by the equation

$$T(t) = J \frac{d\omega_m(t)}{dt} + \omega_o \tag{6}$$

Since the motor is assumed to start from rest, it will be assumed that $\omega_o = 0$.

2.2 Laplace Transform of the DC Motor Model

To analyze the DC motor model in the frequency domain, we can apply the Laplace transform to equations that were aforementioned before. The Laplace transform of the electrical equation is given by:

$$V_a(s) = L_A s I_a(s) + R_A I_a(s) + V_{\text{rot}}(s) \tag{7}$$

For instance, the back EMF will be considered as a disturbance at the input of the motor plant, which depends on the field current and the angular velocity of the motor:

$$V_{\text{rot}}(s) = G_{fq} I_F \omega_m(s) \tag{8}$$

The mechanical equation can also be transformed into the Laplace domain:

$$T(s) = Js\omega_m(s) + T_L(s) \tag{9}$$

As was done with the back EMF, the load torque will be considered as a disturbance at the output of the motor plant, which depends on the angular velocity of the motor:

$$T_L(s) = B\omega_m(s) \tag{10}$$

Using the Eq. (7), the transfer function of the electrical side of the DC motor can be obtained. The transfer function from the armature voltage to the armature current can be expressed as:

$$\frac{I_a(s)}{V_a(s)} = \frac{1/R_A}{\tau s + 1} \tag{11}$$

where $\tau = L_A/R_A$ is the time constant of the motor.

In the same way, using the expression from Eq. (5), and transforming it into the Laplace domain, the transfer function (or simply a gain) from the armature current to the electrical torque can be expressed as:

$$\frac{T_e(s)}{I_a(s)} = G_{fq}I_F \tag{12}$$

From the Eq. (9), the transfer function from the electrical torque to the angular velocity of the motor can be expressed as:

$$\frac{\omega_m(s)}{T_e(s)} = \frac{1}{Js} \tag{13}$$

The block diagram of the DC motor model can be represented as shown in Fig. 2. The input to the system is the armature voltage $V_a(s)$, and the output is the angular velocity of the motor $\omega_m(s)$.

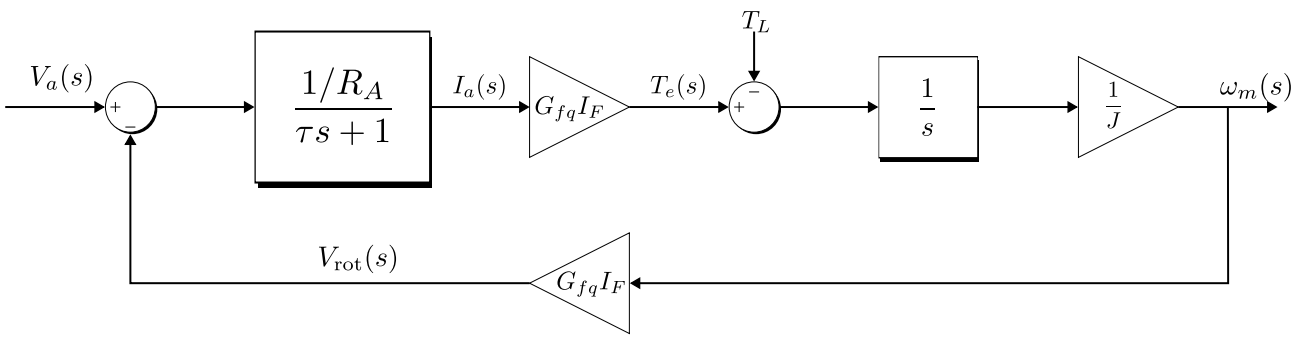


Fig. 2: Block diagram of the DC motor model.

If the load torque is considered linearly depending on the speed, as it shown in Eq. (10), the block diagram can be modified to include the load torque as a disturbance at the output of the system, as shown in Fig. 3.

The DC motor parameters used in this project are summarized in the Table 1. These parameters were chosen to represent a typical 220 V_{rms} DC motor, ensuring that the model captures realistic dynamics and behaviors.

NOTE: For the following sections, the speed is expressed in revolutions per minute (RPM) without loss of generality, as the model can be adapted to other units if necessary. The conversion from angular velocity in radians per second to RPM is given by:

$$\eta_m(t) = \omega_m(t) \cdot \frac{\pi}{30} \tag{14}$$

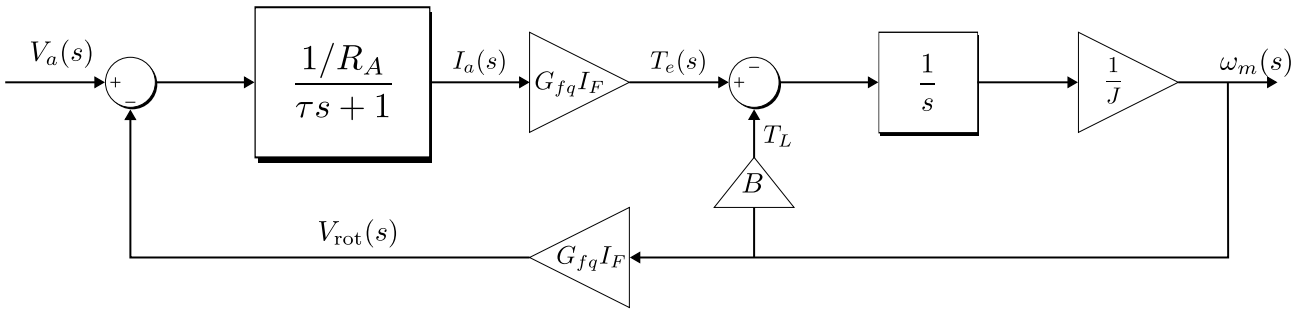


Fig. 3: Block diagram of the DC motor model with load torque.

Table 1: DC Motor Parameters

Parameter	Value
Armature resistance R_A	0.835 Ω
Armature inductance L_A	22 mH
Field current I_F	10 A
Flux linkage constant G_{fq}	0.085 mH
Inertia J	0.2 kgm^2
Load torque constant B	0.5 Nms$/rad$
Maximum armature voltage V_{max}	220 V
Minimum armature voltage V_{min}	-220 V
Sampling time T_s	1 ms
Maximum reference speed η_{max}	1600 RPM
Minimum reference speed η_{min}	0 RPM

3 Data Preparation

The dataset used for training the reinforcement learning controller was generated from a custom MATLAB SIMULINK model of the DC motor. To provide variability for generalization, the simulation included randomized initial conditions for the integrator block and reference velocities.

3.1 Randomization and Initialization

A custom reset function, `localResetFcn`, was used to set:

- Reference velocity $\eta_{\text{ref}} \in [0, 1600]$ RPM
- Initial motor speed $\eta_m(0)$ within $[0.5, 1.5] \cdot \omega_{\text{ref}}$

Fig. 4 shows the randomization space for the initial motor speed and the target reference speed across simulated episodes. This coverage helps ensure that the reinforcement learning agent generalizes to diverse operating conditions, reducing overfitting to a narrow region of the state space.

3.2 Signal Logging and Sampling

The interaction between the controller and a plant is typically discrete in digital control systems. This process involves the acquisition of signals through analog-to-digital converters

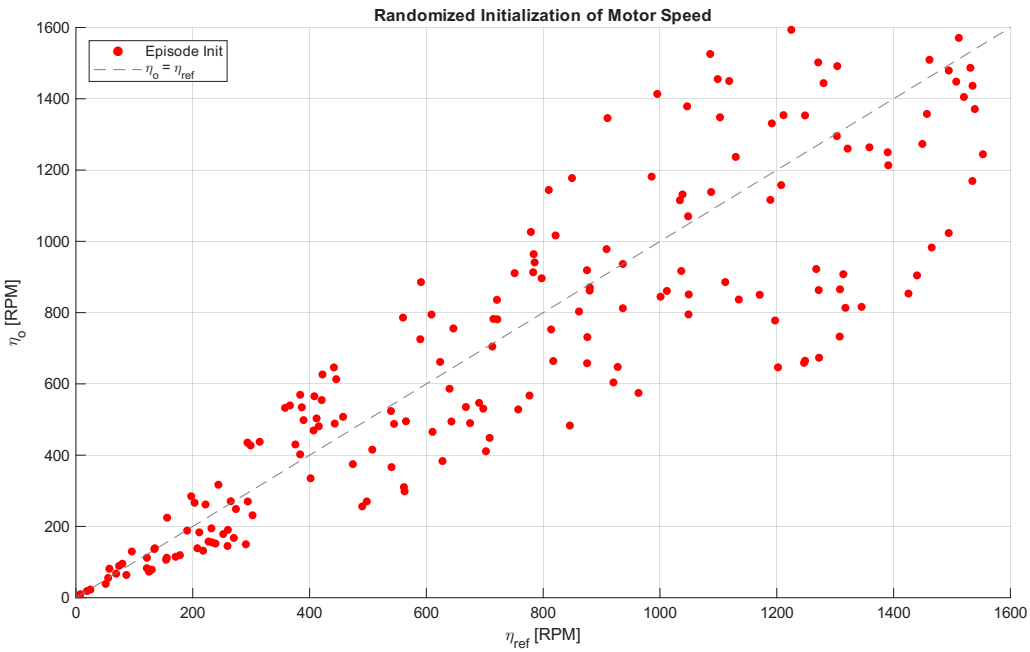


Fig. 4: Distribution of initial motor speed and reference speed across episodes.

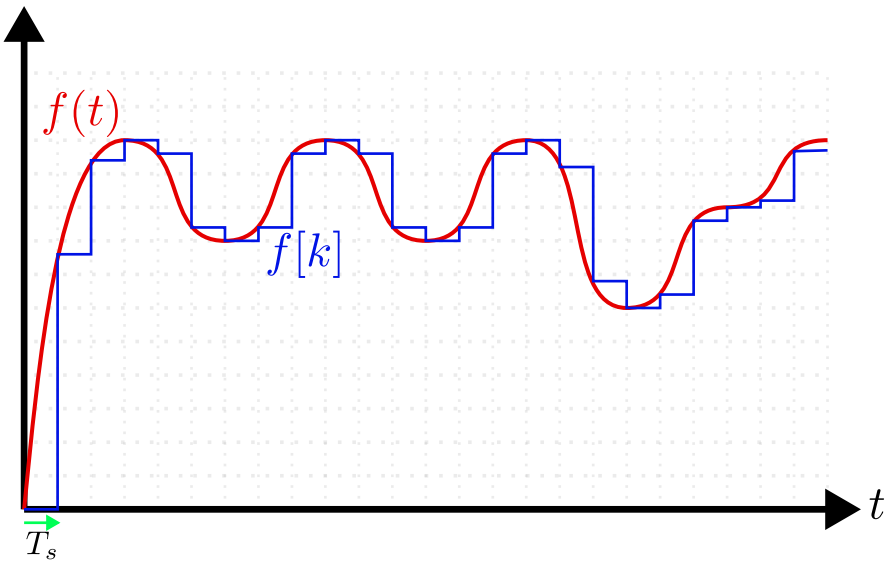


Fig. 5: Example of sampling of a signal $f(t)$. The red line is the original signal, and the blue line is the sampled signal $f[k]$.

(ADC), where they are triggered each sampling time, as shown in the Fig. 5. After processing the signals and applying the defined control low, the controller applies the actions each sample time.

To emulate this real implementation, the DC motor was simulated with a discrete time step of $T_s = 1$ ms, corresponding to a sampling frequency of $f_s = 1$ kHz. This means the controller updates its actions every millisecond. Indeed, this discrete time step governs the resolution at which the agent interacts with the environment and updates its policy.

Therefore, $\omega_m[k]$ is the motor speed at time step k, and $V_a[k]$ is the armature voltage applied at that step.

3.3 Preprocessing

In control systems, it is common to use the integral of the error as part of the state representation. Therefore, the cumulative error was computed as:

$$I_e[k] = \min\left(\max\left(e(k - T_s) + T_s \cdot e(k),\ -V_{\max}\right),\ V_{\max}\right) \tag{15}$$

This integral term is bounded to prevent excessive accumulation, where $V_{\max}$ is the maximum voltage applied to the motor. Using the integral of the error, the state vector $S(t)$ at each time step was defined as:

$$S[k] = \begin{bmatrix} \omega_{\text{ref}}[k] \\ \omega_m[k] \\ I_e[k] \end{bmatrix} \tag{16}$$

The action of the agent ($A[k]$) is the armature voltage applied at time step k, which is a continuous scalar value in the range $[0, 220]$ V.

4 Training and Evaluation

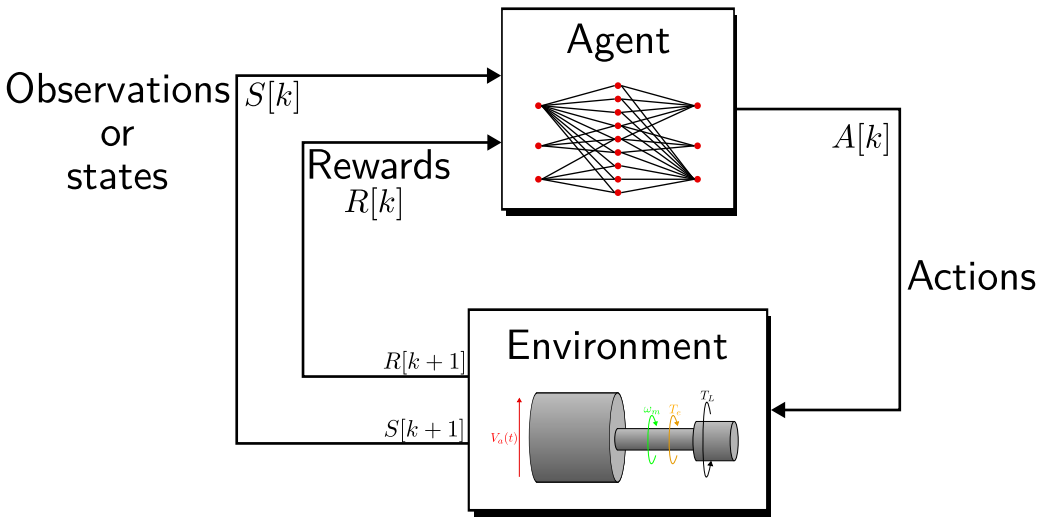


Fig. 6: Reinforcement learning diagram

The control problem was formulated as a reinforcement learning task, where the agent learns to control the DC motor by interacting with the environment. The agent receives observations from the environment, takes actions based on its policy, and receives rewards based on its performance, as is shown in Fig. 6. The training process involved the following steps:

- The agent observes the current state of the environment, represented by the state vector $S[k]$.
- Based on its policy, the agent selects an action $A[k]$. In this case, the action will govern the armature voltage $V_a[k]$ applied to the motor.
- The environment responds by updating the motor speed $\omega_m[k]$ and providing a reward $R[k]$ based on the tracking error.

- The agent updates its policy based on the received reward and the observed state transition.
- This process is repeated for a predefined number of episodes, allowing the agent to learn an optimal policy for controlling the motor.

The training was performed using the deep deterministic policy gradient (DDPG) algorithm with an actor-critic architecture. The environment was defined via `rlSimulinkEnv`, and reset logic was introduced through `localResetFcn`.

To encourage sufficient exploration of the action space, the action output of the policy was perturbed using Ornstein–Uhlenbeck noise. This allowed the agent to discover control strategies that generalize across different initial conditions and reference profiles.

4.1 Reward Function

The scalar reward function at each time step penalized deviation from the reference:

$$R[k] = -|\omega_{\text{ref}}(k) - \omega_m(k)| \tag{17}$$

4.2 Reinforcement Learning as a Markov Decision Process

This closed-loop structure corresponds to a Markov Decision Process (MDP), where the state includes both reference and current speed, and the reward function guides the agent toward minimal steady-state and transient error.

The action space $\mathcal{A}$ is the set of possible armature voltages $V_a[k]$, treated as a continuous scalar. The agent learns a deterministic policy $\pi : \mathcal{S} \to \mathcal{A}$ to minimize tracking error. The transition model is not known a priori but is embedded in the system dynamics simulated in SIMULINK.

The reward function $R[k]$ penalizes deviations from the desired state. The Markov property is assumed to hold: the next state depends only on the current state and action, not on prior history.

This abstraction aligns with the agent design framework discussed in the course, where intelligent decision-making under uncertainty is guided by a value function and updated via observed rewards.

4.3 Learning Agent Architecture

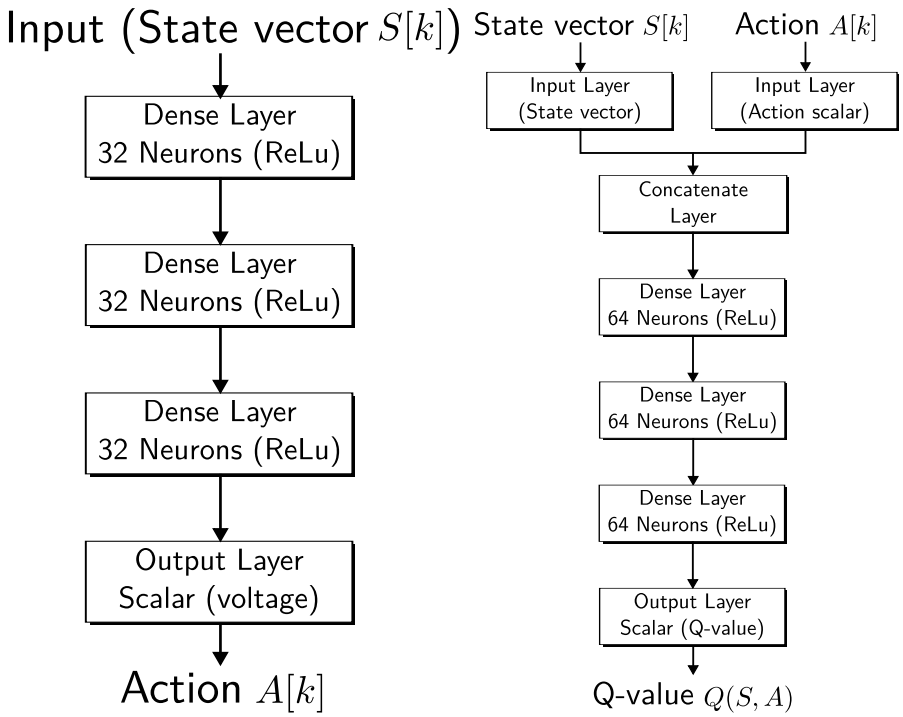
The implemented controller corresponds to a utility-based learning agent [4]. This architecture decomposes into four functional components:

- **Performance element:** a deep neural network (DNN) policy (actor) outputs the control action, and a critic network evaluates its expected value.
- **Learning element:** the policy and value function are updated through temporal-difference learning with mini-batch gradient descent, using past experience stored in a replay buffer.

- **Critic:** provides feedback to the actor based on the Bellman error.
- **Problem generator:** exploration is introduced via noise (Ornstein–Uhlenbeck process [5]) to encourage policy improvement under unseen conditions.

This structure matches the "learning agent" model from the course: an autonomous agent perceiving a dynamic environment, adapting based on feedback, and improving performance over time. The agent is not hard-coded with control laws but discovers them through interaction, generalizing across randomized references and initial states.

4.4 Network Architectures



(a) Actor Network (b) Critic Network

Fig. 7: Neural network architectures

The actor network takes the state vector $S[k]$ as input and passes it through three fully connected hidden layers, each containing 32 neurons with ReLU activation functions, as it shown in the Fig. 7.a). The output layer provides a single scalar action representing the armature voltage command applied to the DC motor. In the other hand, the critic network takes both the state vector $S[k]$ and the action $A[k]$ as inputs. It concatenates these inputs and passes them through three hidden layers with 64 neurons each, also using ReLU activation functions, as it shown in the Fig. 7.b). The output layer produces a scalar Q-value representing the expected return for the given state-action pair.

This can be summarized as follows:

Actor network:

- Three fully connected layers with 32 ReLU neurons each
- Output layer: scalar action (voltage command)

Critic network:

- Dual input (state and action) structure
- Three hidden layers of 64 ReLU neurons after concatenation
- Output layer: scalar Q-value

4.5 Hyperparameters

The performance and stability of deep reinforcement learning algorithms depend heavily on the choice of hyperparameters. These parameters influence the learning dynamics, balance exploration with exploitation, and affect the quality and speed of convergence to an optimal policy. In this project, the hyperparameters were carefully selected based on empirical trials and established best practices from the literature to ensure robust and efficient training of both actor and critic networks. The specific hyperparameters used for training the reinforcement learning controller are listed below:

- Discount factor: $\gamma = 0.8$
- Mini-batch size: 512
- Replay buffer size: 10^6 transitions
- Learning rates: $3 \cdot 10^{-4}$ (actor), $2 \cdot 10^{-4}$ (critic)
- Noise: Ornstein-Uhlenbeck with $\sigma = 0.3$ decaying per episode

4.6 Training Configuration and Process

The training process for the DDPG agent involved a comprehensive setup to ensure effective learning and stable convergence. Training was conducted over 3000 episodes, each comprising up to 3000 time steps, allowing the agent extensive interactions with the simulation environment to develop a robust policy. To systematically track and evaluate learning progress, the agent's state was automatically saved every 100 episodes via a custom callback function, facilitating analysis of intermediate performance and enabling model selection based on performance metrics.

To enhance computational efficiency and accelerate the learning process, parallel training was employed, significantly reducing the overall training duration. Fig. *8* depicts the evolution of the reward signal across training episodes, showing both individual episode rewards and their moving average. The smooth and steadily rising trend of average rewards demonstrates successful policy optimization and highlights the algorithm's effectiveness in progressively minimizing tracking errors through iterative interactions with the environment.

The training was conducted using a custom MATLAB SIMULINK model, as shown in Fig. *9*. This model simulates the DC motor dynamics and provides the necessary environment for the reinforcement learning agent to interact with. The model includes blocks for the reference speed generation, and reward calculation. Also, it includes the RL agent block, which will be trained to control the armature voltage applied to the motor.

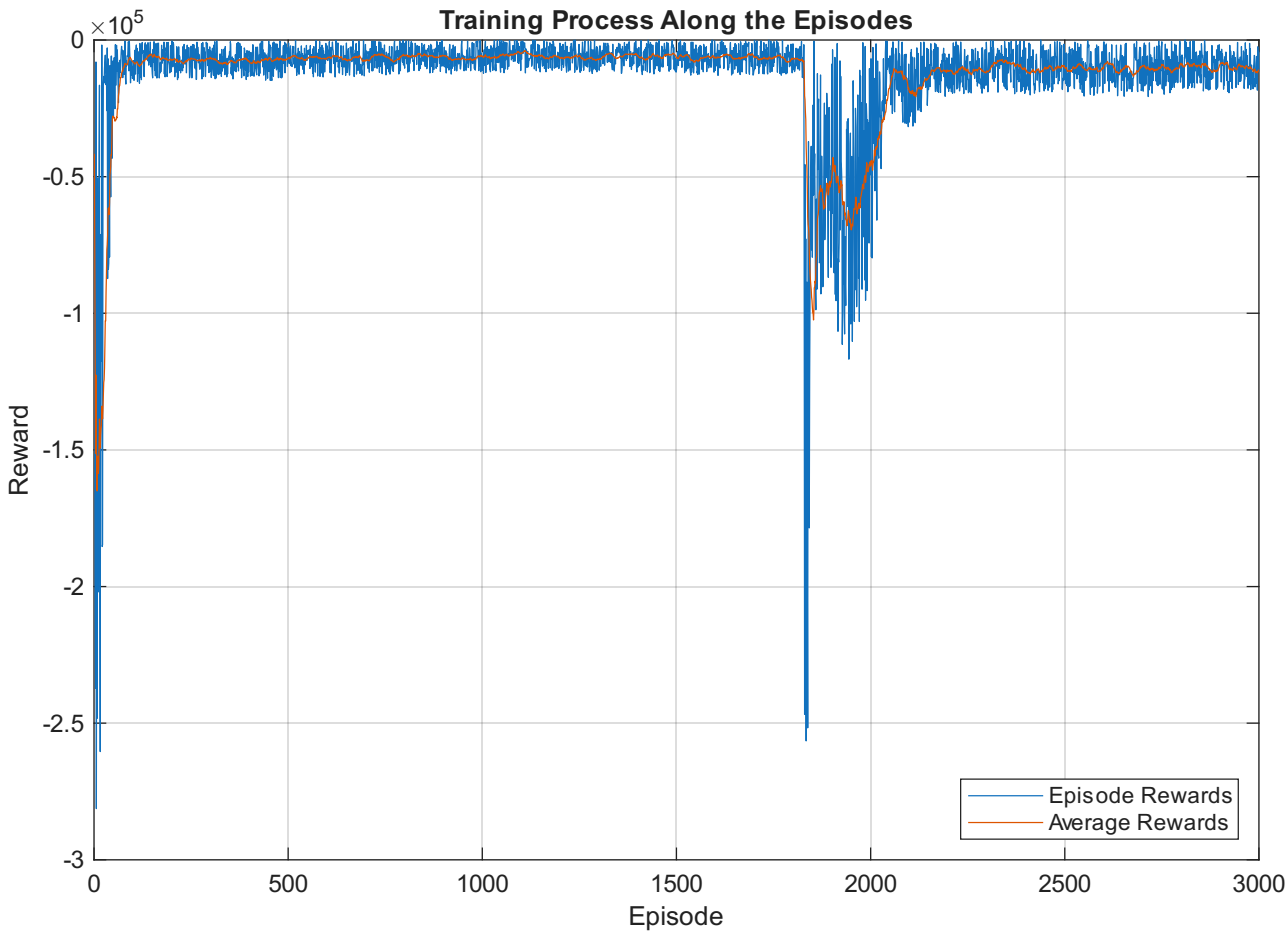


Fig. 8: Training process along the episodes.

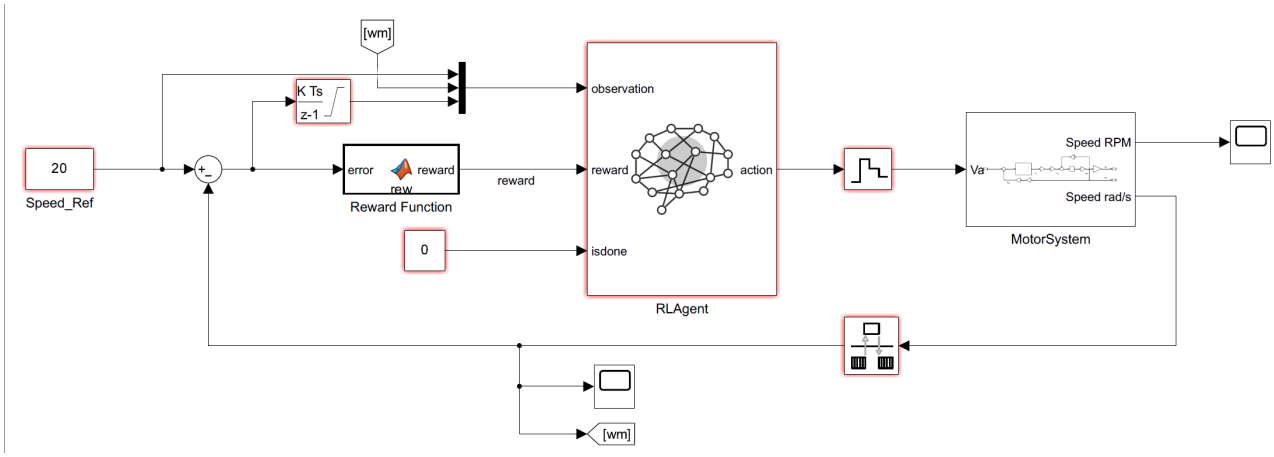


Fig. 9: Schematic of the MATLAB SIMULINK model used for training the DDPG agent.

5 Results

The performance of the trained DDPG agent was thoroughly evaluated by analyzing its capability to follow a dynamic reference angular velocity profile. Specifically, the reference angular velocity was composed of step functions changing instantaneously at $t = 0\,s$, $t = 1\,s$, and $t = 2\,s$.

5.1 Tracking Performance

The DDPG agent demonstrated robust tracking behavior, accurately and swiftly adjusting its control output to follow the reference angular velocity, as illustrated in Fig. *10*. The observed motor speed closely matched the reference profile, exhibiting minimal overshoot at transition instants and achieving low steady-state errors. This indicates the agent successfully generalized its learned policy to manage rapid setpoint changes, showcasing strong transient performance and consistent steady-state accuracy.

Additionally, despite the randomized initial conditions, the agent consistently stabilized the motor's speed, reflecting effective training and adequate exploration during the learning phase. These results highlight the strength of reinforcement learning in handling nonlinear dynamics and uncertainties inherent to DC motor systems.

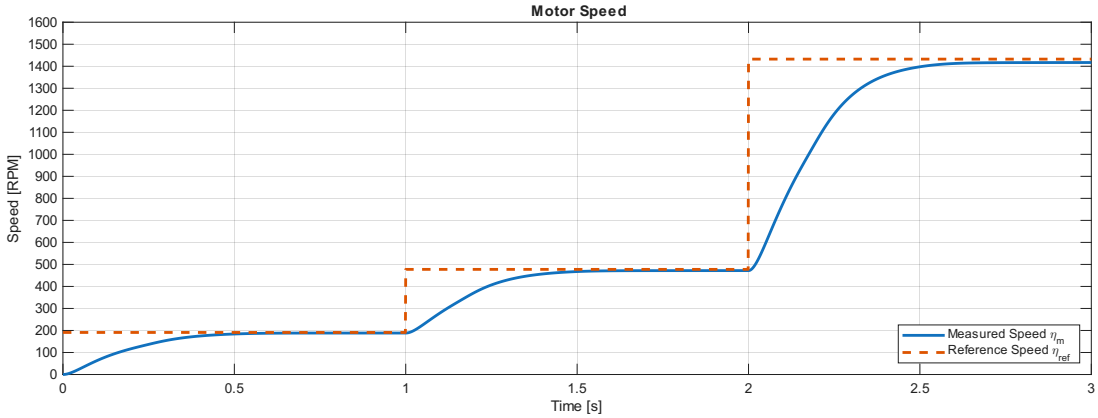
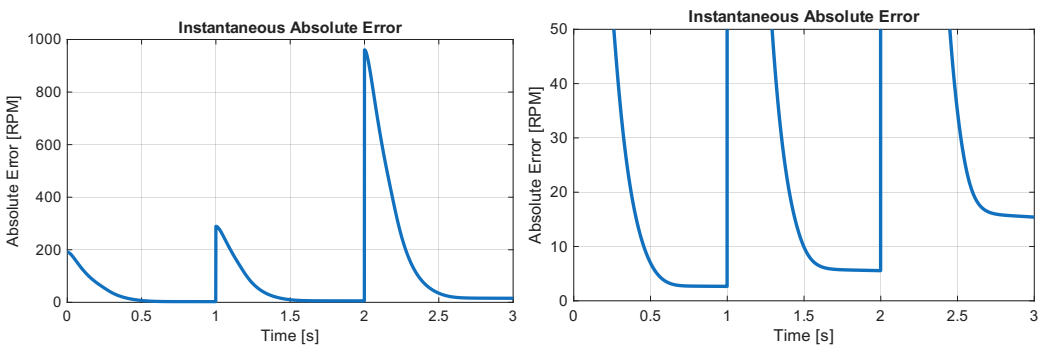


Fig. 10: Motor speed tracking performance.

In order to quantify the tracking performance, the absolute tracking error was computed as the difference between the reference angular velocity and the current motor speed. The absolute error is defined as:

$$e_{\text{abs}}(t) = |\eta_{\text{ref}}(t) - \eta_m(t)| \tag{18}$$



(a) General view. (b) Magnitude zoomed view.

Fig. 11: Absolute error of the motor speed tracking.

From the Fig. *10* and Fig. *11*, it can be observed that the absolute tracking error remained consistently low throughout the evaluation period, with an increasing trend while the reference speed was increasing. As a future work, this trend could be further analyzed to determine if it is due to the limitations of the model or if it can be improved by fine-tuning the hyperparameters or modifying the reward function. The settling time for the motor speed to reach the reference value was approximately 0.6 s for each step change in the reference speed.

5.2 Control Behavior

The control voltage $V_a(t)$ applied by the DDPG agent remained smooth and was reliably confined within the permissible operational bounds of $[0, 220]$ V, as seen in Fig. 12. The gradual transitions in voltage, even during sharp changes in reference velocity, demonstrate that the learned policy effectively avoided aggressive or erratic control actions, thus preventing potential mechanical stress and increasing the longevity and reliability of the motor system. This smooth behavior is advantageous, indicating that the agent effectively balanced rapid responsiveness and cautious voltage control, mitigating potential actuator saturation and instability issues.

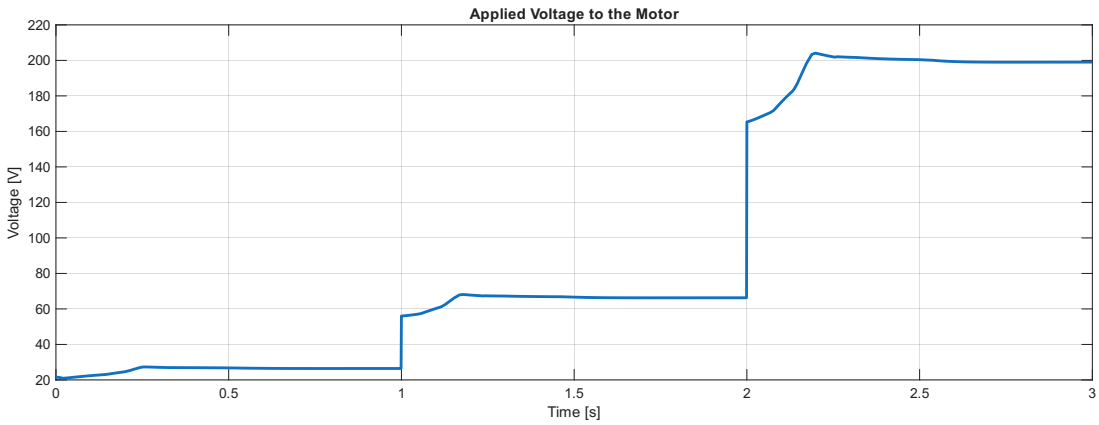


Fig. 12: Armature voltage applied by the RL controller.

5.3 Reward Curve

The reward curve, presented in Fig. 13, illustrates a clear and steady increase throughout the training process, signaling consistent improvement in the agent's ability to minimize tracking error. The progressive growth of reward values over episodes confirms that the DDPG algorithm successfully optimized the policy to minimize deviation from the desired velocity profile. The diminishing magnitude of negative rewards suggests that the agent steadily improved its policy, reducing the average tracking error significantly by the end of training. The smooth convergence pattern further emphasizes the stability and robustness of the reinforcement learning framework adopted in this project.

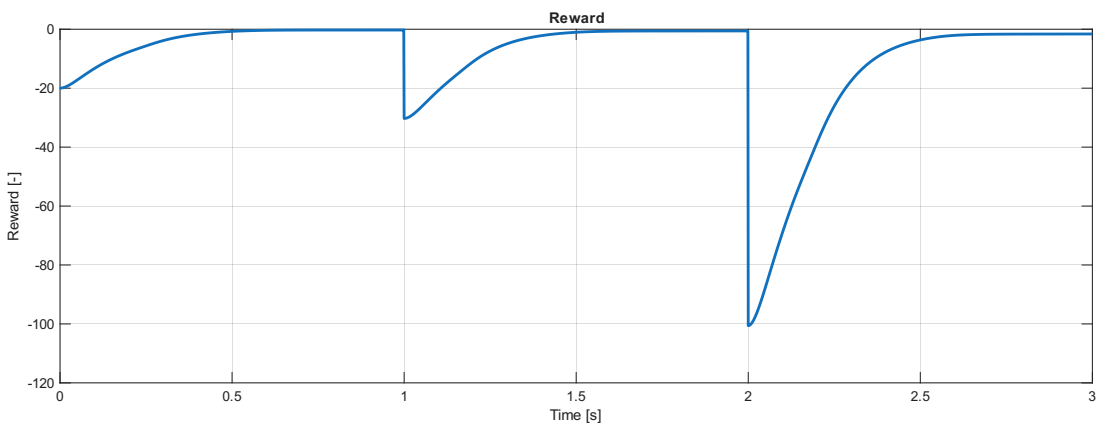


Fig. 13: Reward curve during the simulation.

6 Conclusion

In this report, it was demonstrated that the DDPG algorithm is an effective and robust method for controlling DC motors in dynamic and uncertain conditions. The trained agent consistently exhibited accurate tracking performance with minimal overshoot and low steady-state error, even under varying reference velocities and randomized initial conditions. Furthermore, it was observed that the applied control voltages were smooth and consistently bounded within practical operational limits, ensuring mechanical stability and reliability.

The reward curve clearly illustrated progressive improvement, confirming that the agent successfully learned an optimal control policy through RL. This smooth convergence pattern underscored the stability and effectiveness of the chosen training methodology and hyperparameter settings.

Overall, the findings from this project provide strong support for the adoption of RL techniques in practical control scenarios, highlighting their advantages over traditional methods in terms of adaptability, robustness, and overall performance.

References

[1] S. Umans, A. Fitzgerald, and C. Kingsley, *Electric Machinery: Seventh Edition*. McGraw-Hill, 2013.

[2] S. Sul, *Control of Electric Machine Drive Systems*. IEEE Press Series on Power and Energy Systems, Wiley, 2011.

[3] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep Reinforcement Learning: A Brief Survey," *IEEE Signal Processing Magazine*, vol. 34, pp. 26–38, Nov. 2017.

[4] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," July 2019.

[5] G. E. Uhlenbeck and L. S. Ornstein, "On the theory of the brownian motion," *Phys. Rev.*, vol. 36, pp. 823–841, Sep 1930.